

Estrutura de Dados

Inteligência Artificial e Ciência de Dados | 2026/1



Prof Vitor

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



Algoritmos de busca

- O objetivo desta aula é compreender como funcionam os algoritmos de busca linear e busca binária, identificando:
 - como cada um realiza a busca
 - em que situação cada um pode ser usado
 - por que suas complexidades são diferentes

O problema

Dada a lista:

```
[4, 8, 15, 16, 23, 42]
```

Como descobrir se o valor **23** está nela?

Olhar um por um?

- Ideia:
 - começar no índice 0
 - comparar com o valor procurado
 - se não for igual, avançar
 - repetir até encontrar ou terminar

```
[4, 8, 15, 16, 23, 42]
```

Busca linear

- Na **busca linear**, o algoritmo percorre a lista do início ao fim, comparando cada elemento com o valor procurado.
- Pode acontecer:
 - encontrar no começo
 - encontrar no meio
 - encontrar no final
 - não encontrar



Vamos implementar?

```
def busca_linear(lista, valor):  
    i = 0  
  
    while i < len(lista):  
        if lista[i] == valor:  
            return True  
        i += 1  
  
    return False
```



Complexidade da busca linear

- Melhor caso: $O(1)$
- Pior caso: $O(n)$

Limitação da busca linear

- Em listas muito grandes:
 - pode fazer muitas comparações
 - pode demorar mais do que o necessário
 - não aproveita nenhuma organização da lista

Um novo problema

Dada a lista:

```
[2, 5, 8, 12, 16, 21, 30, 37, 45]
```

Como descobrir se o valor **21** está nela?

Busca binária

- Passos:
 - olhar o elemento do meio
 - comparar com o valor procurado
 - decidir se procura à esquerda ou à direita
 - repetir o processo na metade restante

```
[2, 5, 8, 12, 16, 21, 30, 37, 45]
```

Busca binária

- A **busca binária exige**:
 - lista ordenada
- Se **a lista não estiver ordenada**:
 - o raciocínio quebra
 - o algoritmo pode errar



Vamos implementar?



```
def busca_binaria(lista, valor):  
    inicio = 0  
    fim = len(lista) - 1  
  
    while inicio <= fim:  
        meio = (inicio + fim) // 2  
  
        if lista[meio] == valor:  
            return True  
        elif valor < lista[meio]:  
            fim = meio - 1  
        else:  
            inicio = meio + 1  
  
    return False
```



Complexidade da busca binária

- Melhor caso: $O(1)$
- Pior caso: $O(\log n)$

Obrigado!



Quer trocar mais uma ideia? Me chama!

vhlopes@senacrs.com.br